

Laporan Tugas Kecil 3 IF2211 Strategi Algoritma

Ice Sliding Puzzle Solver

13524051@std.stei.itb.ac.id

mharisputras.work@gmail.com

13524053@std.stei.itb.ac.id

mikhael.yonatan10@gmail.com



Disusun oleh:

Mikhael Andrian Yonatan	13524051
Muhammad Haris Putra Sulastianto	13524053

**Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026**

Daftar Isi

1	Pendahuluan	6
1.1	Deskripsi Singkat Ice Sliding Puzzle	6
2	Representasi Permainan	7
2.1	Representasi Papan	7
2.2	Aturan Permainan	7
2.3	Format Input	8
2.4	Format Output	8
2.5	Definisi State	9
2.6	Definisi Move / Sliding	9
3	Algoritma Pathfinding yang Digunakan	10
3.1	Uniform Cost Search (UCS)	10
3.1.1	Cara Kerja	12
3.1.2	Struktur Data	12
3.1.3	Perhitungan Cost	12
3.1.4	Kompleksitas	13
3.2	Greedy Best First Search (GBFS)	13
3.2.1	Cara Kerja	15
3.2.2	Fungsi Heuristik	15
3.2.3	Prioritas Node	15
3.2.4	Kompleksitas	16
3.3	A* Search	16
3.3.1	Cara Kerja	17
3.3.2	Definisi $g(n)$	18
3.3.3	Definisi $h(n)$	18
3.3.4	Definisi $f(n)$	18
3.3.5	Kompleksitas	19
3.4	Algoritma Iterative Deepening A* (Bonus)	19
3.4.1	Cara Kerja	21
3.4.2	Implementasi pada Program	22
3.4.3	Kelebihan IDA*	23

3.4.4	Kekurangan IDA*	23
3.4.5	Kompleksitas	23
4	Heuristik	24
4.1	Heuristik Utama	24
4.1.1	Manhattan to Goal	24
4.2	Heuristik Alternatif (Bonus)	25
4.2.1	Manhattan to Next Target	25
4.2.2	Manhattan Total to Goal	26
5	Implementasi Program	28
5.1	Bahasa Pemrograman	28
5.2	Struktur Folder Program	28
5.3	Struktur Class dan Module	29
5.3.1	Class State	29
5.3.2	Class Board	29
5.3.3	Class Algoritma	30
5.3.4	Class Heuristic	30
5.4	Cara Membaca File	31
5.5	Validasi Input	31
5.6	Penyimpanan Solusi	32
5.7	Playback System	32
5.8	Graphical User Interface (GUI)	33
6	Analisis Algoritma	33
6.1	Definisi $g(n)$, $h(n)$, dan $f(n)$	33
6.1.1	Definisi $g(n)$	33
6.1.2	Definisi $h(n)$	34
6.1.3	Definisi $f(n)$	34
6.2	Analisis Admissible Heuristic	34
6.3	Perbandingan Algoritma	35
6.3.1	Uniform Cost Search (UCS)	35
6.3.2	Greedy Best First Search (GBFS)	36
6.3.3	A* Search	36

6.3.4	IDA*	36
6.4	Perbandingan Teoritis	37
6.5	Kesimpulan Analisis	37
7	Hasil Percobaan dan Pengujian	38
7.1	Testcase 1 - Papan Tidak Valid 1	38
7.1.1	Input 1	38
7.1.2	Output CLI	38
7.1.3	Output GUI	39
7.2	Testcase 2 - Papan Tidak Valid 2	39
7.2.1	Input 2	39
7.2.2	Output CLI	39
7.2.3	Output GUI	40
7.3	Testcase 3 - Papan Kecil	40
7.3.1	Input 1	40
7.3.2	Output UCS	41
7.3.3	Output GBFS-H1	41
7.3.4	Output GBFS-H2	42
7.3.5	Output GBFS-H3	42
7.3.6	Output A*-H1	43
7.3.7	Output A*-H2	43
7.3.8	Output A*-H3	44
7.3.9	Output IDA*-H1	44
7.3.10	Output IDA*-H2	45
7.3.11	Output IDA*-H3	45
7.3.12	Perbandingan Hasil	46
7.3.13	Analisis	46
7.4	Testcase 4 - Papan Sedang	46
7.4.1	Input 4	46
7.4.2	Output UCS	47
7.4.3	Output GBFS-H1	47
7.4.4	Output GBFS-H2	48
7.4.5	Output GBFS-H3	48

7.4.6	Output A*-H1	49
7.4.7	Output A*-H2	49
7.4.8	Output A*-H3	50
7.4.9	Output IDA*-H1	50
7.4.10	Output IDA*-H2	51
7.4.11	Output IDA*-H3	51
7.4.12	Perbandingan Hasil	52
7.4.13	Analisis	52
7.5	Testcase 5 - Papan Besar	52
7.5.1	Input 5	52
7.5.2	Output UCS	53
7.5.3	Output GBFS-H1	54
7.5.4	Output GBFS-H2	54
7.5.5	Output GBFS-H3	55
7.5.6	Output A*-H1	55
7.5.7	Output A*-H2	56
7.5.8	Output A*-H3	56
7.5.9	Output IDA*-H1	57
7.5.10	Output IDA*-H2	57
7.5.11	Output IDA*-H3	58
7.5.12	Perbandingan Hasil	58
7.5.13	Analisis	58
7.6	Testcase 6 - Impossible Case	59
7.6.1	Input 6	59
7.6.2	Output UCS	59
7.6.3	Output GBFS-H1	60
7.6.4	Output GBFS-H2	60
7.6.5	Output GBFS-H3	61
7.6.6	Output A*-H1	61
7.6.7	Output A*-H2	62
7.6.8	Output A*-H3	62
7.6.9	Output IDA*-H1	63
7.6.10	Output IDA*-H2	63

7.6.11	Output IDA*-H3	64
7.6.12	Analisis	64
7.7	Analisis Akhir	64
A	Lampiran	65
A.1	Link Repository	65
A.2	Pernyataan	65
A.3	Tabel Spesifikasi Tugas Kecil	66

1 Pendahuluan

1.1 Deskripsi Singkat Ice Sliding Puzzle

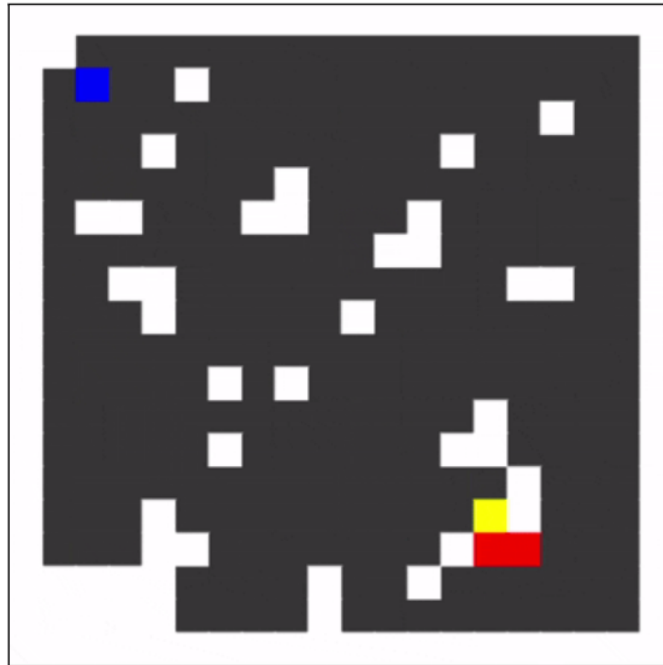


Figure 1: Visualisasi Permainan Ice Sliding Puzzle

Ice Sliding Puzzle merupakan permainan berbasis grid yang mengharuskan pemain membawa aktor dari posisi awal menuju titik tujuan pada permukaan es yang licin. Aktor hanya dapat bergerak ke empat arah utama, yaitu atas, bawah, kiri, dan kanan. Namun, ketika suatu arah dipilih, aktor akan terus meluncur hingga menabrak rintangan atau dinding sehingga tidak dapat berhenti secara bebas pada setiap petak.

Pada tugas ini, permainan dimodifikasi dengan penambahan beberapa aturan khusus. Papan permainan dapat terdiri atas berbagai jenis tile, seperti rintangan, lava, titik tujuan, serta tile angka yang harus dilewati secara berurutan. Selain itu, setiap tile memiliki traversal cost yang memengaruhi total biaya solusi. Oleh karena itu, permasalahan ini dapat dimodelkan sebagai permasalahan pathfinding dengan constraint tambahan.

Untuk menyelesaikan permasalahan tersebut, digunakan beberapa algoritma pencarian jalur, yaitu Uniform Cost Search (UCS), Greedy Best First Search (GBFS), dan A* Search. Setiap algoritma memiliki strategi pencarian dan karakteristik performa yang berbeda.

2 Representasi Permainan

2.1 Representasi Papan

Permainan direpresentasikan dalam bentuk grid dua dimensi berukuran $N \times M$, dengan N menyatakan jumlah baris dan M menyatakan jumlah kolom. Setiap sel pada papan merepresentasikan suatu tile dengan fungsi tertentu dalam permainan.

Karakter yang digunakan pada papan dijelaskan pada Tabel 1.

Table 1: Representasi Tile pada Papan

Simbol	Keterangan
Z	Posisi awal aktor
0	Titik tujuan
*	Tile yang dapat dilewati
X	Rintangan atau batu
L	Lava (game over jika dilewati)
0-9	Tile angka yang harus dilewati berurutan

Selain papan utama, program juga menggunakan matriks traversal cost yang memiliki ukuran sama dengan papan permainan. Setiap elemen pada matriks cost menyatakan biaya yang dikenakan ketika aktor melewati tile tersebut.

2.2 Aturan Permainan

Permainan memiliki beberapa aturan utama sebagai berikut:

1. Aktor hanya dapat bergerak ke empat arah, yaitu atas, bawah, kiri, dan kanan.
2. Ketika suatu arah dipilih, aktor akan terus meluncur hingga:
 - menabrak dinding,
 - menabrak rintangan (X),
 - atau berhenti tepat sebelum tile penghalang.
3. Jika aktor bergerak menuju sisi papan tanpa adanya penghalang, maka aktor keluar dari papan dan permainan dianggap gagal.
4. Tile lava (L) tidak boleh dilewati. Jalur yang melewati lava dianggap tidak valid.
5. Tile angka harus dilewati secara berurutan mulai dari 0, 1, dan seterusnya. Jika urutan dilanggar, permainan dianggap gagal.
6. Setelah tile angka berhasil dilewati sesuai urutan, tile tersebut berubah menjadi tile biasa dan dapat dilewati kembali tanpa constraint tambahan.

7. Permainan dianggap selesai apabila:

- seluruh tile angka telah dilewati sesuai urutan, dan
- aktor berhenti tepat pada tile tujuan (0).

2.3 Format Input

Input program diberikan dalam bentuk file `.txt`. Format input terdiri atas dua bagian utama:

1. Dimensi papan permainan.
2. Representasi papan dan traversal cost.

Baris pertama berisi dua bilangan bulat N dan M yang menyatakan jumlah baris dan kolom papan.

Selanjutnya terdapat:

- N baris representasi papan permainan.
- N baris traversal cost untuk setiap tile.

Contoh format input:

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999
```

2.4 Format Output

Program menghasilkan beberapa keluaran utama, yaitu:

1. Solusi gerakan yang ditemukan.

2. Total cost solusi.
3. Visualisasi papan pada setiap langkah solusi.
4. Jumlah iterasi atau node yang diperiksa.
5. Waktu eksekusi algoritma.
6. Playback langkah penyelesaian permainan.

Contoh solusi:

```
Path      : RULUDRUR
Total Cost : 87
Iterasi    : 20
Waktu     : 9 ms
```

2.5 Definisi State

State merepresentasikan kondisi permainan pada suatu waktu tertentu. Setiap state terdiri atas:

1. Posisi aktor pada papan.
2. Progress angka yang telah dilewati.
3. Cost total yang telah dikeluarkan.
4. Riwayat langkah menuju state tersebut.
5. Arah gerak terakhir untuk mencapai state saat ini.
6. Penanda apakah state saat ini berhasil atau gagal (game over).
7. Referensi ke state sebelumnya

State digunakan sebagai node pada algoritma pathfinding.

2.6 Definisi Move / Sliding

Move merupakan aksi perpindahan aktor ke salah satu arah utama:

$$\{U, D, L, R\}$$

dengan:

- $U = \text{Up}$,

- D = Down,
- L = Left,
- R = Right.

Berbeda dengan pergerakan biasa pada grid, satu move pada permainan ini menyebabkan aktor terus bergerak hingga mencapai titik berhenti valid. Oleh karena itu, satu aksi dapat melewati beberapa tile sekaligus.

Cost suatu move dihitung sebagai jumlah traversal cost seluruh tile yang dilalui selama proses sliding, tidak termasuk tile awal sebelum bergerak, namun termasuk tile tempat aktor berhenti.

Jika selama sliding aktor:

- melewati lava,
- keluar papan,
- atau melanggar urutan angka,

maka move dianggap tidak valid dan state tujuan tidak akan dibangkitkan.

3 Algoritma Pathfinding yang Digunakan

Program menggunakan beberapa algoritma pathfinding untuk mencari solusi Ice Sliding Puzzle, yaitu UCS, GBFS, A*, dan IDA*.

3.1 Uniform Cost Search (UCS)

```

1 package solver.algorithm;
2
3 import solver.core.*;
4 import solver.engine.*;
5
6 import java.util.*;
7
8 public class UCS implements SearchAlgorithm {
9
10     @Override
11     public SearchResult solve(Board board) {
12
13         PriorityQueue<State> pq = new PriorityQueue<>(Comparator.
14             comparingInt(s -> s.gCost));
15         Set<VisitedKey> visited = new HashSet<>();
16         List<State> history = new ArrayList<>();
17
18         State start = new State(board.startX, board.startY);
19         pq.add(start);

```

```

19
20     int iterations = 0;
21
22     while (!pq.isEmpty()) {
23         State current = pq.poll();
24         iterations++;
25
26         VisitedKey key = new VisitedKey(current.x, current.y,
27             current.nextNumber);
28
29         if (visited.contains(key))
30             continue;
31         visited.add(key);
32
33         history.add(new State(current));
34
35         // GOAL CHECK
36         if (current.x == board.goalX &&
37             current.y == board.goaly &&
38             current.nextNumber == board.maxNumber + 1) {
39
40             return new SearchResult(
41                 true,
42                 current.gCost,
43                 current.path,
44                 iterations,
45                 history);
46
47             // EXPAND
48             for (Direction dir : Direction.values()) {
49                 Optional<State> next = Movement.slide(board,
50                     current, dir);
51
52                 if (next.isPresent()) {
53                     State nextState = next.get();
54                     nextState.parent = current;
55                     nextState.lastDir = dir;
56                     pq.add(nextState);
57                 } else {
58                     State deadState = new State(current, dir);
59                     history.add(deadState);
60                 }
61             }
62
63             return new SearchResult(false, -1, "", iterations, history)
64                 ;
65     }

```

Lihat file UCS.java pada repository (lihat Lampiran A.1).

Uniform Cost Search (UCS) merupakan algoritma pencarian berbasis cost yang selalu memilih node dengan total cost aktual terkecil untuk dieksplorasi terlebih dahulu. Secara teoritis, UCS dapat menghasilkan solusi optimal apabila seluruh

edge cost bernilai non-negatif.

3.1.1 Cara Kerja

1. Node awal dimasukkan ke dalam priority queue dengan cost awal 0.
2. Node dengan total cost terkecil diambil dari priority queue.
3. Dilakukan pengecekan apakah node tersebut merupakan goal state.
4. Jika belum mencapai goal, seluruh kemungkinan move dibangkitkan untuk menghasilkan successor state.
5. Cost baru dihitung berdasarkan traversal cost sepanjang proses sliding.
6. Successor state dimasukkan ke priority queue selama state tersebut belum pernah diproses sebelumnya.
7. Proses diulang hingga goal ditemukan atau tidak ada node tersisa.

3.1.2 Struktur Data

Algoritma UCS menggunakan beberapa struktur data utama:

- **Priority Queue**
Digunakan untuk mengambil node dengan total cost terkecil.
- **Visited Map / Cost Map**
Digunakan untuk menyimpan cost minimum yang pernah ditemukan pada setiap state.
- **Parent Tracking**
Digunakan untuk merekonstruksi jalur solusi setelah goal ditemukan.

3.1.3 Perhitungan Cost

Pada UCS, prioritas node ditentukan berdasarkan total traversal cost aktual yang telah ditempuh dari start menuju node tersebut.

$$g(n) = \sum \text{cost seluruh tile yang dilalui}$$

Cost dihitung dari seluruh tile yang dilewati selama sliding, termasuk tile akhir tempat berhenti, namun tidak termasuk tile awal sebelum bergerak.

3.1.4 Kompleksitas

Secara teoritis, kompleksitas UCS adalah:

$$O(b^d)$$

dengan:

- b = branching factor,
- d = kedalaman solusi optimal.

Karena menggunakan priority queue, operasi insertion dan extraction memiliki kompleksitas:

$$O(\log n)$$

UCS memiliki jaminan optimalitas, tetapi penggunaan memori dapat menjadi besar karena seluruh frontier harus disimpan.

3.2 Greedy Best First Search (GBFS)

```
1 package solver.algorithm;
2
3 import solver.core.*;
4 import solver.engine.*;
5 import solver.algorithm.heuristic.Heuristic;
6
7 import java.util.*;
8
9 public class GBFS implements SearchAlgorithm {
10
11     private final Heuristic heuristic;
12
13     public GBFS(Heuristic heuristic) {
14         this.heuristic = heuristic;
15     }
16
17     @Override
18     public SearchResult solve(Board board) {
19
20         PriorityQueue<State> pq = new PriorityQueue<>(
21             Comparator.comparingInt(s -> heuristic.estimate(s,
22                 board)));
23
24         Set<VisitedKey> visited = new HashSet<>();
25         List<State> history = new ArrayList<>();
26
27         State start = new State(board.startX, board.startY);
28         pq.add(start);
```

```

28
29     int iterations = 0;
30
31     while (!pq.isEmpty()) {
32         State current = pq.poll();
33         iterations++;
34
35         VisitedKey key = new VisitedKey(current.x, current.y,
36             current.nextNumber);
37
38         if (visited.contains(key))
39             continue;
40         visited.add(key);
41
42         history.add(new State(current));
43
44         // GOAL
45         if (current.x == board.goalX &&
46             current.y == board.goaly &&
47             current.nextNumber == board.maxNumber + 1) {
48
49             return new SearchResult(
50                 true,
51                 current.gCost,
52                 current.path,
53                 iterations,
54                 history);
55
56             // EXPAND
57             for (Direction dir : Direction.values()) {
58                 Optional<State> next = Movement.slide(board,
59                     current, dir);
60
61                 if (next.isPresent()) {
62                     State nextState = next.get();
63                     nextState.parent = current;
64                     nextState.lastDir = dir;
65                     pq.add(nextState);
66                 } else {
67                     State deadState = new State(current, dir);
68                     history.add(deadState);
69                 }
70             }
71
72             return new SearchResult(false, -1, "", iterations, history)
73                 ;
74     }

```

Lihat file GBFS.java pada repository (lihat Lampiran A.1).

Greedy Best First Search (GBFS) merupakan algoritma pencarian heuristik yang memilih node berdasarkan estimasi kedekatan terhadap goal tanpa mempertimbangkan cost aktual yang telah ditempuh.

3.2.1 Cara Kerja

1. Node awal dimasukkan ke dalam priority queue.
2. Node dengan nilai heuristik terkecil dipilih untuk dieksplorasi.
3. Dilakukan pengecekan apakah node merupakan goal state.
4. Jika belum mencapai goal, successor state dibangkitkan dari seluruh move yang valid.
5. Nilai heuristik setiap successor dihitung.
6. Successor dimasukkan ke priority queue berdasarkan prioritas heuristik terkecil.
7. Proses diulang hingga solusi ditemukan atau node habis.

3.2.2 Fungsi Heuristik

GBFS menggunakan heuristic function:

$$h(n)$$

yang merepresentasikan estimasi jarak node saat ini menuju goal.

Contoh heuristik yang digunakan:

- Manhattan Distance
- Euclidean Distance
- Jumlah checkpoint angka yang belum dilewati

Node dengan nilai heuristik lebih kecil akan diproses lebih dahulu.

3.2.3 Prioritas Node

Prioritas node pada GBFS ditentukan hanya berdasarkan heuristic function:

$$f(n) = h(n)$$

GBFS tidak mempertimbangkan cost perjalanan sebelumnya sehingga solusi yang dihasilkan belum tentu optimal.

3.2.4 Kompleksitas

Kompleksitas GBFS secara teoritis adalah:

$$O(b^m)$$

dengan:

- b = branching factor,
- m = maksimum kedalaman pencarian.

Pada kasus tertentu, GBFS dapat bekerja sangat cepat apabila heuristik cukup baik. Namun, algoritma ini rentan terjebak pada local optimum dan tidak menjamin solusi optimal.

3.3 A* Search

```
1 package solver.algorithm;
2
3 import solver.core.*;
4 import solver.engine.*;
5 import solver.algorithm.heuristic.Heuristic;
6
7 import java.util.*;
8
9 public class AStar implements SearchAlgorithm {
10
11     private final Heuristic heuristic;
12
13     public AStar(Heuristic heuristic) {
14         this.heuristic = heuristic;
15     }
16
17     @Override
18     public SearchResult solve(Board board) {
19
20         PriorityQueue<State> pq = new PriorityQueue<>(
21             Comparator.comparingInt(s -> s.gCost + heuristic.
22                 estimate(s, board)));
23
24         Set<VisitedKey> visited = new HashSet<>();
25         List<State> history = new ArrayList<>();
26
27         State start = new State(board.startX, board.startY);
28         pq.add(start);
29
30         int iterations = 0;
31
32         while (!pq.isEmpty()) {
33             State current = pq.poll();
34             iterations++;
```

```

34         VisitedKey key = new VisitedKey(current.x, current.y,
35             current.nextNumber);
36
37         if (visited.contains(key))
38             continue;
39         visited.add(key);
40
41         history.add(new State(current));
42
43         // GOAL CHECK
44         if (current.x == board.goalX &&
45             current.y == board.goaly &&
46             current.nextNumber == board.maxNumber + 1) {
47
48             return new SearchResult(
49                 true,
50                 current.gCost,
51                 current.path,
52                 iterations,
53                 history);
54         }
55
56         // EXPAND
57         for (Direction dir : Direction.values()) {
58             Optional<State> next = Movement.slide(board,
59                 current, dir);
60
61             if (next.isPresent()) {
62                 State nextState = next.get();
63                 nextState.parent = current;
64                 nextState.lastDir = dir;
65                 pq.add(nextState);
66             } else {
67                 State deadState = new State(current, dir);
68                 history.add(deadState);
69             }
70         }
71
72         return new SearchResult(false, -1, "", iterations, history)
73         ;
74     }

```

Lihat file AStar.java pada repository (lihat Lampiran A.1).

A* Search merupakan algoritma pathfinding yang menggabungkan actual traversal cost dan heuristic estimate untuk menentukan prioritas node. A* merupakan pengembangan dari UCS dan GBFS.

3.3.1 Cara Kerja

1. Node awal dimasukkan ke dalam priority queue.

2. Node dengan nilai evaluasi terkecil dipilih untuk dieksplorasi.
3. Dilakukan pengecekan apakah node merupakan goal state.
4. Jika belum mencapai goal, successor state dibangkitkan dari seluruh move valid.
5. Untuk setiap successor dihitung:
 - traversal cost dari start,
 - estimasi cost menuju goal.
6. Successor dimasukkan ke priority queue berdasarkan nilai evaluasi total.
7. Proses diulang hingga solusi ditemukan.

3.3.2 Definisi $g(n)$

$$g(n)$$

merupakan total traversal cost nyata dari start node menuju node n .

$$g(n) = \sum \text{cost tile yang telah dilalui}$$

3.3.3 Definisi $h(n)$

$$h(n)$$

merupakan estimasi cost dari node n menuju goal.

Nilai heuristik dapat dihitung menggunakan:

- Manhattan Distance,
- Euclidean Distance,
- estimasi jumlah sliding,
- atau heuristic custom lainnya.

3.3.4 Definisi $f(n)$

Fungsi evaluasi pada A^* didefinisikan sebagai:

$$f(n) = g(n) + h(n)$$

Node dengan nilai $f(n)$ terkecil akan diprioritaskan untuk dieksplorasi.

3.3.5 Kompleksitas

Kompleksitas teoritis A* adalah:

$$O(b^d)$$

Namun performa aktual sangat dipengaruhi oleh kualitas heuristik yang digunakan.

Jika heuristik admissible dan konsisten, maka:

- A* menjamin solusi optimal,
- jumlah node yang dieksplorasi dapat jauh lebih sedikit dibanding UCS.

Kelemahan utama A* adalah penggunaan memori yang besar karena seluruh frontier dan visited state perlu disimpan.

3.4 Algoritma Iterative Deepening A*(Bonus)

```
1 package solver.algorithm;
2
3 import solver.algorithm.heuristic.Heuristic;
4 import solver.core.Board;
5 import solver.core.State;
6 import solver.engine.Direction;
7 import solver.engine.Movement;
8
9 import java.util.ArrayList;
10 import java.util.List;
11 import java.util.Optional;
12
13 public class IDAStar implements SearchAlgorithm {
14     private final Heuristic heuristic;
15     private int iterations;
16
17     public IDAStar(Heuristic heuristic) {
18         this.heuristic = heuristic;
19     }
20
21     @Override
22     public SearchResult solve(Board board) {
23         this.iterations = 0;
24         State start = new State(board.startX, board.startY);
25         int threshold = heuristic.estimate(start, board);
26
27         List<State> currentPath = new ArrayList<>();
28         List<State> searchTrace = new ArrayList<>();
29
30         while (true) {
31             currentPath.clear();
32             searchTrace.clear();
```

```

33         Object result = search(start, threshold, board,
34                                 currentPath, searchTrace);
35
36         if (result instanceof SearchResult) {
37             return (SearchResult) result;
38         }
39
40         int nextThreshold = (Integer) result;
41         if (nextThreshold == Integer.MAX_VALUE) {
42             return new SearchResult(false, 0, "", iterations,
43                                     searchTrace);
44         }
45         threshold = nextThreshold;
46     }
47
48     private Object search(State current, int threshold, Board board
49                           , List<State> currentPath, List<State> searchTrace) {
50         iterations++;
51
52         int f = current.gCost + heuristic.estimate(current, board);
53
54         if (f > threshold) {
55             return f;
56         }
57
58         currentPath.add(current);
59         searchTrace.add(new State(current));
60
61         if (current.x == board.goalX && current.y == board.goalY &&
62             current.nextNumber == board.maxNumber + 1) {
63             return new SearchResult(
64                 true,
65                 current.gCost,
66                 current.path,
67                 iterations,
68                 searchTrace);
69         }
70
71         int min = Integer.MAX_VALUE;
72
73         for (Direction dir : Direction.values()) {
74             Optional<State> next = Movement.slide(board, current,
75                                                     dir);
76
77             if (next.isPresent()) {
78                 State nextState = next.get();
79                 nextState.lastDir = dir;
80                 nextState.parent = current;
81
82                 if (isStateInHistory(nextState, currentPath))
83                     continue;
84
85                 Object res = search(nextState, threshold, board,
86                                     currentPath, searchTrace);

```

```

82         if (res instanceof SearchResult) {
83             return res;
84         }
85
86         int t = (Integer) res;
87         if (t < min) min = t;
88     } else {
89         State deadState = new State(current, dir);
90         searchTrace.add(deadState);
91     }
92 }
93
94 currentPath.remove(currentPath.size() - 1);
95 return min;
96 }
97
98 private boolean isStateInHistory(State next, List<State>
99     history) {
100     for (State s : history) {
101         if (s.x == next.x && s.y == next.y && s.nextNumber ==
102             next.nextNumber) {
103             return true;
104         }
105     }
106     return false;
107 }

```

Lihat file *IDAStar.java* pada repository (lihat Lampiran A.1).

IDA* merupakan algoritma pathfinding yang menggabungkan konsep heuristic search pada A* dengan pendekatan depth-first search (DFS). Berbeda dengan A* yang menyimpan seluruh frontier pada priority queue, IDA* melakukan pencarian secara iteratif menggunakan batas nilai evaluasi tertentu (*threshold*). Pendekatan ini membuat penggunaan memori IDA* lebih kecil dibandingkan A*.

3.4.1 Cara Kerja

IDA* menggunakan fungsi evaluasi:

$$f(n) = g(n) + h(n)$$

dengan:

- $g(n)$ = total traversal cost dari start menuju node n ,
- $h(n)$ = estimasi cost menuju goal menggunakan heuristic function.

Langkah kerja IDA* adalah sebagai berikut:

1. Threshold awal diinisialisasi menggunakan nilai heuristik dari start state.

2. Algoritma melakukan pencarian DFS secara rekursif.
3. Pada setiap node dihitung nilai:

$$f(n) = g(n) + h(n)$$

4. Jika nilai $f(n)$ melebihi threshold saat ini, maka node tidak dieksplorasi lebih lanjut.
5. Selama pencarian, algoritma menyimpan nilai $f(n)$ terkecil yang melebihi threshold.
6. Jika goal belum ditemukan, threshold diperbesar menggunakan nilai minimum tersebut.
7. Proses pencarian diulang hingga solusi ditemukan atau tidak ada node valid tersisa.

3.4.2 Implementasi pada Program

Method utama:

```
solve(Board board)
```

bertugas untuk:

- menginisialisasi threshold awal,
- melakukan iterative deepening,
- menjalankan pencarian DFS berulang.

Proses pencarian rekursif dilakukan pada method:

```
search(...)
```

Method tersebut:

- menghitung nilai $f(n)$,
- melakukan pruning apabila $f(n)$ melebihi threshold,
- membangkitkan successor state,
- mengecek goal state,
- melakukan backtracking.

Untuk mencegah siklus tak hingga, digunakan method:

```
isStateInHistory(...)
```

Method ini memeriksa apakah state yang akan dikunjungi sudah pernah muncul pada jalur DFS saat ini.

3.4.3 Kelebihan IDA*

IDA* memiliki beberapa kelebihan:

1. Penggunaan memori lebih kecil dibanding A* karena menggunakan DFS.
2. Tetap memanfaatkan heuristic function untuk mengarahkan pencarian.
3. Dapat menemukan solusi optimal apabila heuristic bersifat admissible.

3.4.4 Kekurangan IDA*

Kekurangan utama IDA* adalah:

1. Banyak node dapat dieksplorasi ulang pada iterasi threshold berikutnya.
2. Waktu eksekusi dapat lebih lambat dibanding A* pada beberapa kasus.
3. Implementasi rekursif dapat menyebabkan kedalaman stack besar pada map kompleks.

3.4.5 Kompleksitas

Secara teoritis, kompleksitas waktu IDA* mendekati:

$$O(b^d)$$

dengan:

- b = branching factor,
- d = kedalaman solusi.

Namun, penggunaan memorinya jauh lebih kecil dibanding A* karena hanya menyimpan jalur DFS aktif.

Kompleksitas memori IDA* adalah:

$$O(d)$$

karena algoritma hanya menyimpan node pada current search path.

4 Heuristik

Heuristik digunakan untuk membantu algoritma menentukan arah pencarian menuju goal. Pada program ini, seluruh heuristik menggunakan pendekatan Manhattan Distance karena permainan berbasis grid dengan pergerakan empat arah.

Manhattan Distance menghitung jumlah perpindahan horizontal dan vertikal minimum antara dua posisi pada grid.

Secara umum, rumus Manhattan Distance adalah:

$$\text{dist}(A, B) = |x_A - x_B| + |y_A - y_B|$$

Walaupun pergerakan pada permainan bersifat sliding, Manhattan Distance tetap dapat digunakan sebagai estimasi sederhana untuk mengarahkan pencarian menuju target.

4.1 Heuristik Utama

4.1.1 Manhattan to Goal

```
1 package solver.algorithm.heuristic;
2
3 import solver.core.*;
4
5 public class ManhattanGoal implements Heuristic {
6
7     @Override
8     public int estimate(State state, Board board) {
9         return Math.abs(state.x - board.goalX) +
10             Math.abs(state.y - board.goalY);
11     }
12 }
```

Lihat file `ManhattanGoal.java` pada repository (lihat Lampiran A.1).

Heuristik utama yang digunakan pada program ini adalah *Manhattan to Goal*.

Heuristik ini langsung menghitung estimasi jarak dari posisi aktor saat ini menuju titik goal tanpa mempertimbangkan checkpoint angka yang belum dilewati.

Rumus yang digunakan:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

dengan:

- (x_1, y_1) = posisi aktor saat ini,
- (x_2, y_2) = posisi titik goal.

Langkah perhitungan heuristik:

1. Mengambil posisi aktor saat ini.
2. Mengambil posisi titik goal.
3. Menghitung selisih horizontal dan vertikal.
4. Menjumlahkan kedua selisih tersebut untuk memperoleh estimasi jarak.

Heuristik ini dipilih sebagai heuristik utama karena:

- sederhana dan cepat dihitung,
- memiliki kompleksitas rendah,
- cukup efektif untuk mengarahkan pencarian menuju goal.

Namun, heuristik ini belum mempertimbangkan:

- urutan checkpoint angka,
- obstacle,
- maupun mekanisme sliding.

4.2 Heuristik Alternatif (Bonus)

4.2.1 Manhattan to Next Target

```
1 package solver.algorithm.heuristic;
2
3 import solver.core.Board;
4 import solver.core.State;
5
6 public class ManhattanNextTarget implements Heuristic {
7     @Override
8     public int estimate(State state, Board board) {
9         int targetX;
10        int targetY;
11
12        if (state.nextNumber <= board.maxNumber) {
13            int[] coords = board.targetCoords.get(state.nextNumber)
14                ;
15            targetX = coords[0];
16            targetY = coords[1];
17        }
18        else {
19            targetX = board.goalX;
20            targetY = board.goalY;
21        }
22    }
23 }
```

```

21
22         return Math.abs(state.x - targetX) + Math.abs(state.y -
23             targetY);
24     }

```

Lihat file `ManhattanNextTarget.java` pada repository (lihat Lampiran A.1).

Heuristik ini menghitung estimasi jarak menuju target aktif berikutnya.

Target yang dimaksud dapat berupa:

- checkpoint angka berikutnya yang harus dilewati,
- atau titik goal apabila seluruh checkpoint telah selesai.

Rumus yang digunakan:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

dengan:

- (x_1, y_1) = posisi aktor saat ini,
- (x_2, y_2) = posisi target aktif.

Langkah perhitungan heuristik:

1. Mengecek checkpoint berikutnya yang harus dicapai.
2. Jika checkpoint masih tersedia, target ditetapkan pada checkpoint tersebut.
3. Jika seluruh checkpoint selesai, target berubah menjadi goal.
4. Menghitung Manhattan Distance dari posisi aktor menuju target aktif.

Dibanding Manhattan to Goal, heuristik ini lebih memperhatikan urutan objective pada permainan.

4.2.2 Manhattan Total to Goal

```

1 package solver.algorithm.heuristic;
2
3 import solver.core.Board;
4 import solver.core.State;
5
6 public class ManhattanTotal implements Heuristic {
7     @Override
8     public int estimate(State state, Board board) {
9         int totalHeuristic = 0;
10

```

```

11     if (state.nextNumber <= board.maxNumber) {
12         int[] firstTarget = board.targetCoords.get(state.
13             nextNumber);
14         totalHeuristic += Math.abs(state.x - firstTarget[0]) +
15             Math.abs(state.y - firstTarget[1]);
16
17         for (int i = state.nextNumber; i < board.maxNumber; i
18             ++){
19             int[] currentTarget = board.targetCoords.get(i);
20             int[] nextTarget = board.targetCoords.get(i + 1);
21             totalHeuristic += Math.abs(currentTarget[0] -
22                 nextTarget[0]) + Math.abs(currentTarget[1] -
23                 nextTarget[1]);
24         }
25
26         int[] lastTarget = board.targetCoords.get(board.
27             maxNumber);
28         totalHeuristic += Math.abs(lastTarget[0] - board.goalX)
29             + Math.abs(lastTarget[1] - board.goalY);
30     } else {
31         totalHeuristic += Math.abs(state.x - board.goalX) +
32             Math.abs(state.y - board.goalY);
33     }
34
35     return totalHeuristic;
36 }

```

Lihat file *ManhattanTotal.java* pada repository (lihat Lampiran A.1).

Heuristik ini memperkirakan total jarak yang masih harus ditempuh hingga permainan selesai.

Perhitungan dilakukan dengan menjumlahkan:

- jarak dari posisi aktor menuju checkpoint berikutnya,
- jarak antar checkpoint yang belum dikunjungi,
- serta jarak dari checkpoint terakhir menuju goal.

Rumus yang digunakan:

$$\text{dist}(A, B) = |x_A - x_B| + |y_A - y_B|$$

$$h(n) = \text{dist}(P_{curr}, P_k) + \sum_{i=k}^{N-1} \text{dist}(P_i, P_{i+1}) + \text{dist}(P_N, P_{goal})$$

dengan:

- P_{curr} = posisi aktor saat ini,

- P_k = checkpoint berikutnya,
- P_N = checkpoint terakhir,
- P_{goal} = posisi goal.

Langkah perhitungan heuristik:

1. Menghitung jarak dari posisi aktor menuju checkpoint berikutnya.
2. Menghitung total jarak antar seluruh checkpoint yang belum dikunjungi.
3. Menghitung jarak dari checkpoint terakhir menuju goal.
4. Menjumlahkan seluruh nilai tersebut menjadi estimasi total cost.

Heuristik ini memberikan estimasi yang lebih lengkap dibanding dua heuristik sebelumnya karena mempertimbangkan keseluruhan urutan checkpoint hingga goal.

5 Implementasi Program

5.1 Bahasa Pemrograman

Program dikembangkan menggunakan bahasa pemrograman Java dengan paradigma Object-Oriented Programming (OOP). Pemilihan Java dilakukan karena memiliki:

- dukungan struktur data yang lengkap,
- manajemen memori otomatis,
- portabilitas tinggi,
- serta kemudahan implementasi GUI dan modularisasi program.

Program dikompilasi menggunakan Java Development Kit (JDK) dan dapat dijalankan pada sistem operasi Windows maupun Linux.

5.2 Struktur Folder Program

- `algorithm`
Berisi implementasi algoritma pathfinding dan heuristic function.
- `core`
Berisi representasi state, board, tile, dan struktur data utama permainan.
- `engine`
Mengatur mekanisme movement dan direction.

- **parser**
Bertanggung jawab membaca serta memvalidasi file input.
- **utils**
Berisi utility tambahan seperti logger dan timer.
- **visualization**
Berisi sistem visualisasi CLI dan GUI.
- **Main.java**
Titik awal eksekusi program.

5.3 Struktur Class dan Module

Struktur program dibagi menjadi beberapa class sesuai fungsi masing-masing.

5.3.1 Class State

Class **State** digunakan untuk merepresentasikan node pada proses pencarian.

Atribut utama:

- **x, y**
Posisi aktor pada papan.
- **nextNumber**
Menyimpan checkpoint angka berikutnya yang harus dilewati.
- **gCost**
Total traversal cost dari start node menuju state saat ini.
- **path**
String yang menyimpan urutan gerakan solusi.
- **lastDir**
Arah gerakan terakhir.
- **isGameOver**
Menandai apakah state merupakan kondisi gagal.
- **parent**
Pointer menuju parent state untuk rekonstruksi solusi.

5.3.2 Class Board

Class **Board** merepresentasikan papan permainan beserta traversal cost setiap tile.

Class ini bertanggung jawab untuk:

- menyimpan grid permainan,
- menyimpan matrix cost,
- menyediakan akses tile,
- mengecek validitas posisi.

5.3.3 Class Algoritma

Setiap algoritma pencarian diimplementasikan dalam class terpisah:

- UCS.java
- GBFS.java
- AStar.java
- IDAStar.java

Seluruh algoritma mengimplementasikan interface:

`SearchAlgorithm`

Dengan interface yang sama, algoritma baru dapat ditambahkan tanpa mengubah alur utama program.

5.3.4 Class Heuristic

Sistem heuristik dibuat menggunakan interface:

`Heuristic`

dengan method:

```
int estimate(State state, Board board)
```

Implementasi heuristik:

- ManhattanGoal
- ManhattanNextTarget
- ManhattanTotal

5.4 Cara Membaca File

Pembacaan file dilakukan oleh class:

`InputParser.java`

Tahapan pembacaan file:

1. Membaca ukuran papan $N \times M$.
2. Membaca representasi grid permainan.
3. Membaca traversal cost setiap tile.
4. Membentuk object `Board`.
5. Mengidentifikasi posisi awal dan goal.

Jika terjadi kesalahan format input, parser akan melempar exception dan program menampilkan pesan error.

5.5 Validasi Input

Program melakukan validasi terhadap file masukan untuk memastikan konfigurasi permainan valid.

Validasi yang dilakukan meliputi:

- ukuran board sesuai dengan jumlah baris dan kolom,
- karakter tile valid,
- terdapat tepat satu tile start (Z),
- terdapat tepat satu tile goal (O),
- jumlah traversal cost sesuai ukuran board,
- angka checkpoint tersusun berurutan.

Jika input tidak valid, program akan menghentikan proses pencarian dan menampilkan pesan kesalahan.

5.6 Penyimpanan Solusi

Penyimpanan solusi dilakukan menggunakan class:

`Logger.java`

Informasi yang disimpan:

- path solusi,
- total cost,
- jumlah iterasi,
- waktu eksekusi,
- visualisasi langkah solusi.

File solusi disimpan dalam format `.txt` sehingga mudah dibaca kembali oleh pengguna.

5.7 Playback System

Program dapat menampilkan kembali solusi secara bertahap melalui fitur playback.

Playback CLI diimplementasikan pada:

`PlaybackCLI.java`

Sistem playback bekerja dengan:

1. menyimpan seluruh state solusi,
2. menampilkan board pada setiap langkah,
3. memungkinkan navigasi maju dan mundur.

Visualisasi state dilakukan menggunakan:

`RendererCLI.java`

Setiap state dicetak ulang sehingga pengguna dapat melihat proses perpindahan aktor dari awal hingga mencapai goal.

5.8 Graphical User Interface (GUI)

Sebagai implementasi bonus, program juga menyediakan GUI untuk memvisualisasikan permainan.

Komponen GUI berada pada folder:

`visualization/gui/`

Class utama GUI:

- `MainWindow.java`
- `BoardPanel.java`
- `ControlPanel.java`
- `PlaybackController.java`

GUI digunakan untuk menampilkan papan permainan, menjalankan playback, serta mempermudah pemilihan algoritma dan heuristik.

6 Analisis Algoritma

Bagian ini membahas analisis teoritis dari algoritma pathfinding yang digunakan pada program, yaitu Uniform Cost Search (UCS), Greedy Best First Search (GBFS), A* Search, dan IDA*. Analisis dilakukan terhadap fungsi evaluasi, admissibility heuristik, optimality, completeness, kompleksitas waktu, serta penggunaan memori.

6.1 Definisi $g(n)$, $h(n)$, dan $f(n)$

Pada algoritma pencarian heuristik, setiap node memiliki nilai evaluasi tertentu yang digunakan untuk menentukan prioritas eksplorasi.

6.1.1 Definisi $g(n)$

$$g(n)$$

merepresentasikan total traversal cost nyata dari start node menuju node n .

Pada implementasi program ini, cost dihitung dari seluruh tile yang dilalui selama proses sliding.

$$g(n) = \sum \text{cost tile yang dilalui}$$

Semakin besar traversal cost yang telah ditempuh, semakin besar nilai $g(n)$.

6.1.2 Definisi $h(n)$

$$h(n)$$

merupakan estimasi cost dari node n menuju goal.

Program menggunakan heuristic function berbasis Manhattan Distance untuk memperkirakan jarak menuju:

- checkpoint berikutnya,
- atau goal apabila seluruh checkpoint telah dilewati.

Nilai heuristic membantu algoritma menentukan arah pencarian yang dianggap paling menjanjikan.

6.1.3 Definisi $f(n)$

$$f(n)$$

merupakan fungsi evaluasi total yang digunakan pada algoritma heuristic.

Pada A* dan IDA*:

$$f(n) = g(n) + h(n)$$

dengan:

- $g(n)$ = traversal cost aktual,
- $h(n)$ = estimasi cost menuju goal.

Node dengan nilai $f(n)$ terkecil akan diprioritaskan untuk dieksplorasi.

6.2 Analisis Admissible Heuristic

Agar algoritma A* dapat menjamin solusi optimal, heuristic function yang digunakan harus bersifat admissible.

Heuristik disebut admissible apabila:

$$h(n) \leq h^*(n)$$

dengan:

- $h(n)$ = estimasi heuristic,
- $h^*(n)$ = cost minimum sebenarnya menuju goal.

Pada implementasi ini, heuristik Manhattan Distance dapat dianggap admissible karena:

- hanya menghitung estimasi jarak minimum,
- tidak melebihi-lebihkan cost sebenarnya,
- mengabaikan obstacle dan constraint tambahan.

Walaupun mekanisme sliding menyebabkan jalur aktual dapat lebih panjang dibanding estimasi Manhattan Distance, nilai heuristik tetap tidak melampaui cost sebenarnya menuju goal.

Akibatnya:

$$h(n) \leq h^*(n)$$

sehingga heuristik tetap memenuhi sifat admissible.

Namun, heuristik ini belum sepenuhnya akurat karena:

- tidak mempertimbangkan posisi obstacle,
- tidak mempertimbangkan arah sliding,
- tidak memperhitungkan traversal cost tiap tile.

6.3 Perbandingan Algoritma

6.3.1 Uniform Cost Search (UCS)

UCS mengeksplorasi node berdasarkan traversal cost terkecil yang telah ditempuh.

Kelebihan UCS:

- menjamin solusi optimal,
- tidak membutuhkan heuristic function.

Kekurangan UCS:

- jumlah node yang dieksplorasi dapat sangat besar,
- penggunaan memori tinggi karena seluruh frontier disimpan.

6.3.2 Greedy Best First Search (GBFS)

GBFS menggunakan heuristic function untuk memilih node yang dianggap paling dekat dengan goal.

Kelebihan GBFS:

- pencarian dapat berjalan lebih cepat,
- jumlah eksplorasi node umumnya lebih sedikit.

Kekurangan GBFS:

- tidak menjamin solusi optimal,
- dapat terjebak pada local optimum,
- sangat bergantung pada kualitas heuristik.

6.3.3 A* Search

A* menggabungkan traversal cost dan heuristic estimate pada fungsi evaluasi:

$$f(n) = g(n) + h(n)$$

Kelebihan A*:

- menjamin solusi optimal apabila heuristik admissible,
- pencarian lebih terarah dibanding UCS.

Kekurangan A*:

- penggunaan memori besar,
- performa sangat dipengaruhi kualitas heuristik.

6.3.4 IDA*

IDA* menggunakan evaluasi heuristik seperti A*, tetapi melakukan pencarian secara depth-first dengan threshold tertentu.

Kelebihan IDA*:

- penggunaan memori lebih kecil dibanding A*,
- tetap memanfaatkan heuristic function.

Kekurangan IDA*:

- banyak node dapat dieksplorasi ulang,
- waktu eksekusi dapat lebih lama dibanding A*.

6.4 Perbandingan Teoritis

Table 2: Perbandingan Karakteristik Algoritma

Algoritma	Optimal	Complete	Waktu	Memori
UCS	Ya	Ya	Tinggi	Tinggi
GBFS	Tidak	Tidak selalu	Rendah	Sedang
A*	Ya*	Ya	Sedang	Tinggi
IDA*	Ya*	Ya	Sedang-Tinggi	Rendah

Keterangan:

- A* dan IDA* optimal apabila heuristic function bersifat admissible.
- GBFS dapat bekerja sangat cepat, tetapi kualitas solusi sangat bergantung pada heuristic yang digunakan.
- UCS memiliki eksplorasi paling luas karena tidak menggunakan heuristic.
- IDA* lebih hemat memori dibanding A* karena menggunakan pendekatan DFS.

6.5 Kesimpulan Analisis

Berdasarkan analisis teoritis:

- UCS paling stabil dalam menghasilkan solusi optimal, tetapi eksplorasinya besar.
- GBFS memiliki pencarian tercepat pada beberapa kasus, namun solusi yang ditemukan belum tentu optimal.
- A* memberikan keseimbangan terbaik antara optimalitas dan efisiensi pencarian.
- IDA* lebih efisien dari sisi memori, tetapi dapat melakukan eksplorasi ulang pada banyak state.

Pada permainan Ice Sliding Puzzle, penggunaan heuristik membantu mengurangi jumlah eksplorasi state secara signifikan dibanding pencarian tanpa heuristic.

7 Hasil Percobaan dan Pengujian

Pengujian dilakukan menggunakan beberapa jenis testcase dengan karakteristik berbeda. Pada laporan ini, hanya akan ditampilkan perbandingan hasil dan output dalam mode GUI dari setiap algoritma dan heuristiknya. Output akhir, baik mode CLI dan GUI akan menghasilkan file dengan ekstensi txt yang berisi statistik dari solusi beserta langkah-langkah yang menuju solusi saja. Untuk melihat langkah-langkah pencarian secara menyeluruh, dapat dilihat pada log.

7.1 Testcase 1 - Papan Tidak Valid 1

7.1.1 Input 1

```
7 7
XXXXXXX
X0****X
X**X**X
X****OX
X1**LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3   5   2   8   1   999
999 7   4   999 6   9   999
999 2   8   3   5   4   999
999 6   1   7   2   999 999
999 9   3   4   999 8   999
999 999 999 999 999 999 999
```

7.1.2 Output CLI

```
=====ICE SLIDING PUZZLE SOLVER=====
Masukkan nama file input (contoh: map1.txt): input1.txt
[ERROR] Gagal membaca file: Invalid row length at row 4
```

Figure 2: Hasil eksekusi CLI testcase 1

7.1.3 Output GUI

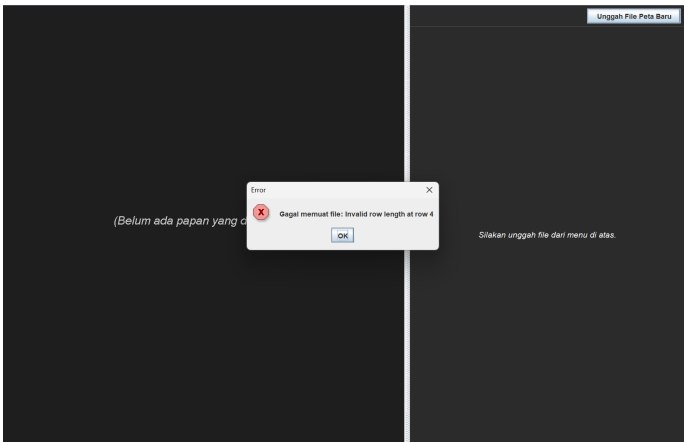


Figure 3: Hasil eksekusi GUI testcase 1

7.2 Testcase 2 - Papan Tidak Valid 2

7.2.1 Input 2

```
7 7
XXXXXXX
X*****X
X**X**X
X****OX
X1***LX
XZ**X*X
XXXXXXX
999 999 999 999 999 999 999
999 3 5 2 8 1 999
999 7 4 999 6 9 999
999 2 8 3 5 4 999
999 6 1 7 2 999 999
999 9 3 4 999 8 999
999 999 999 999 999 999 999
```

7.2.2 Output CLI



Figure 4: Hasil eksekusi CLI testcase 2

7.2.3 Output GUI

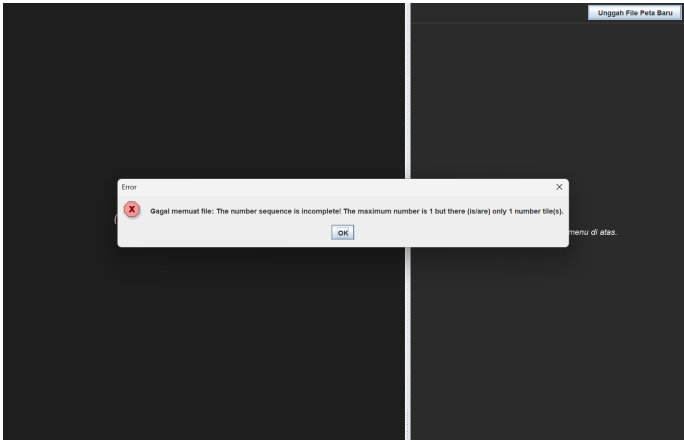


Figure 5: Hasil eksekusi GUI testcase 2

7.3 Testcase 3 - Papan Kecil

7.3.1 Input 1

```
5 5
XXXXX
XZ*OX
X*X*X
X***X
XXXXX
999 999 999 999 999
999 1    50  1    999
999 2    999 2    999
999 2    1    1    999
999 999 999 999 999
```

7.3.2 Output UCS

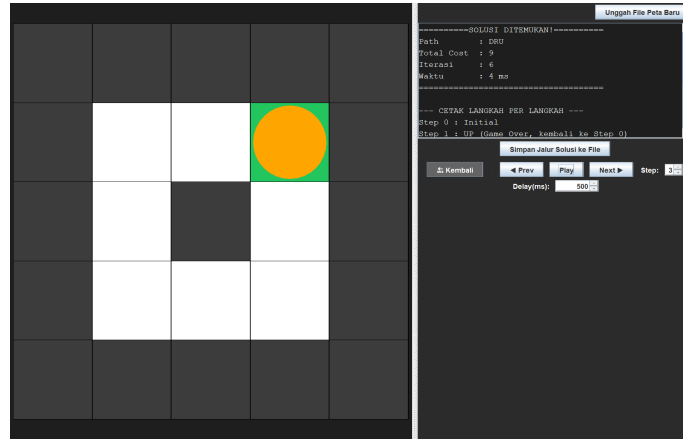


Figure 6: Hasil eksekusi UCS testcase 3

7.3.3 Output GBFS-H1

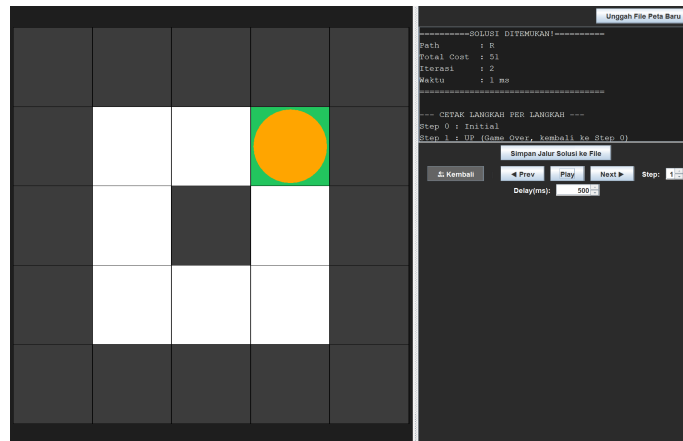


Figure 7: Hasil eksekusi GBFS-H1 testcase 3

7.3.4 Output GBFS-H2

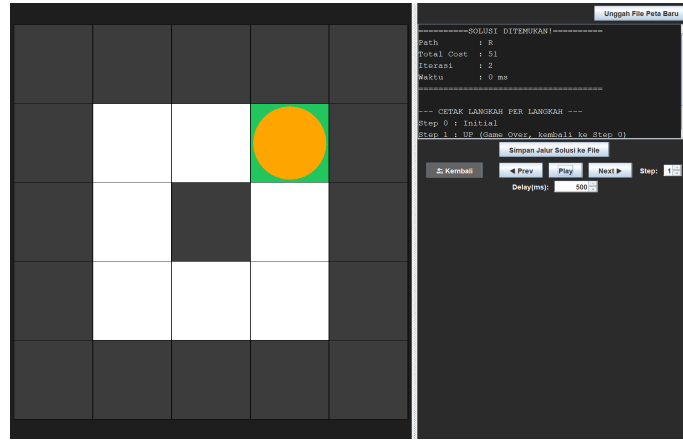


Figure 8: Hasil eksekusi GBFS-H2 testcase 3

7.3.5 Output GBFS-H3

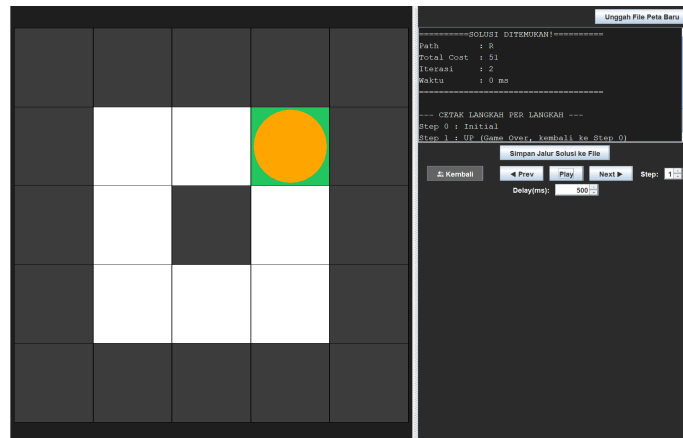


Figure 9: Hasil eksekusi GBFS-H3 testcase 3

7.3.6 Output A*-H1

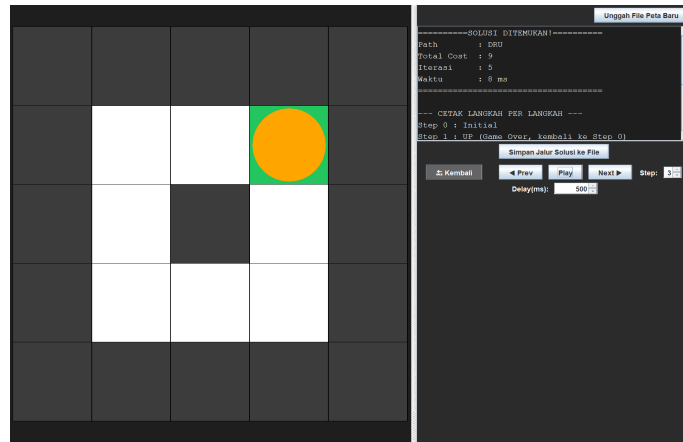


Figure 10: Hasil eksekusi A*-H1 testcase 3

7.3.7 Output A*-H2

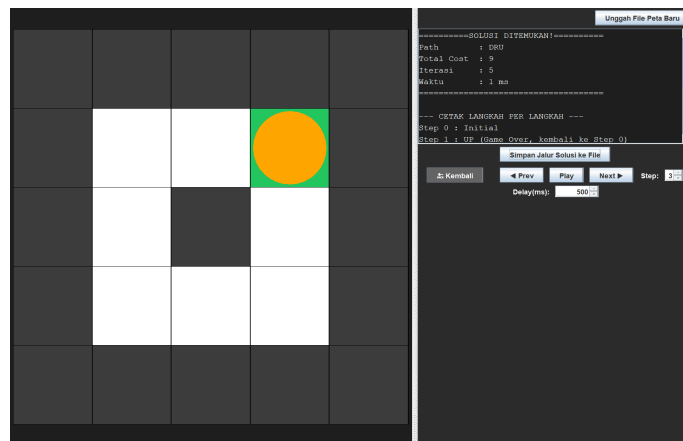


Figure 11: Hasil eksekusi A*-H2 testcase 3

7.3.8 Output A*-H3

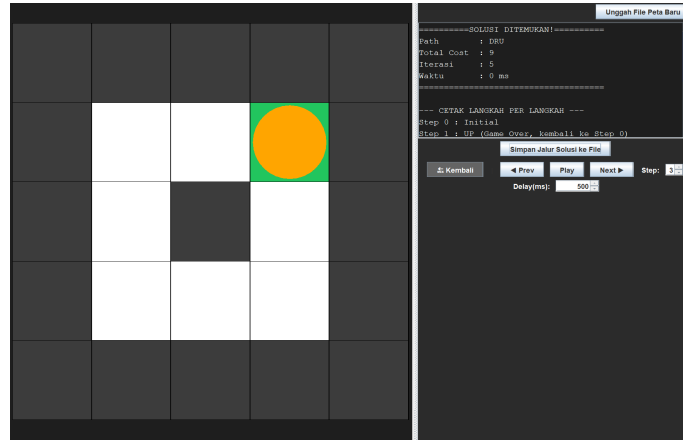


Figure 12: Hasil eksekusi A*-H3 testcase 3

7.3.9 Output IDA*-H1

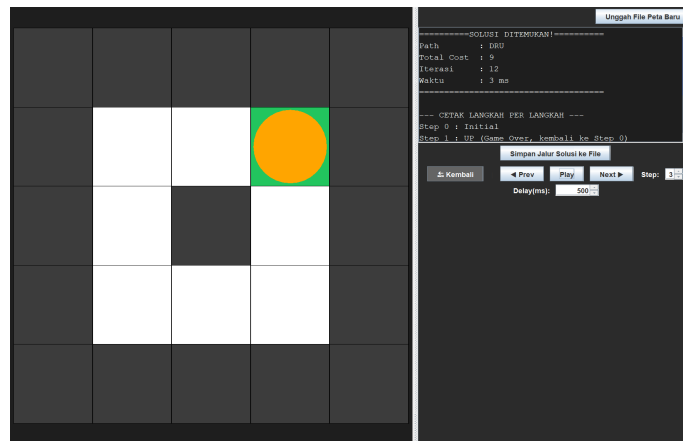


Figure 13: Hasil eksekusi IDA*-H1 testcase 3

7.3.10 Output IDA*-H2

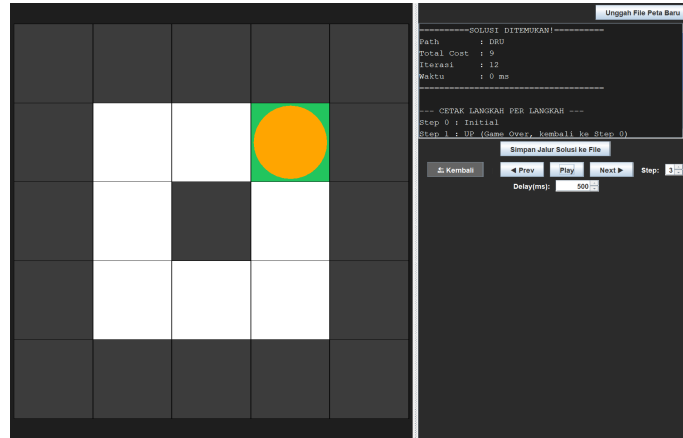


Figure 14: Hasil eksekusi IDA*-H2 testcase 3

7.3.11 Output IDA*-H3

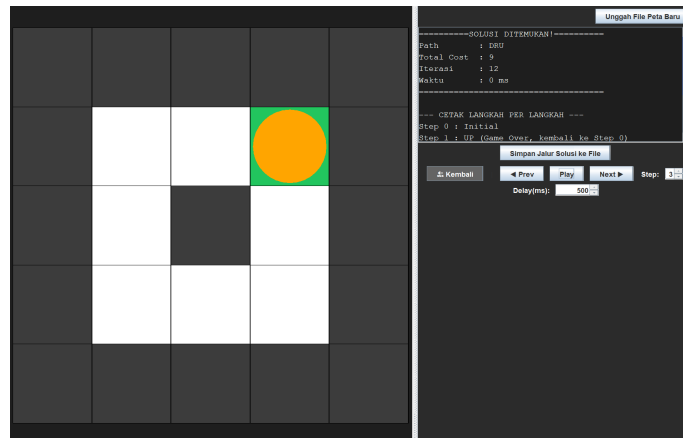


Figure 15: Hasil eksekusi IDA*-H3 testcase 3

7.3.12 Perbandingan Hasil

Table 3: Hasil Pengujian TC3

Algoritma	Heuristik	Cost	Iterasi	Waktu (ms)
UCS	-	9	6	4
GBFS	H1	51	2	1
GBFS	H2	51	2	0..
GBFS	H3	51	2	0..
A*	H1	9	5	8
A*	H2	9	5	1
A*	H3	9	5	0..
IDA*	H1	9	12	3
IDA*	H2	9	12	0..
IDA*	H3	9	12	0..

7.3.13 Analisis

Berdasarkan hasil pengujian, algoritma GBFS terbukti menghasilkan rute yang tidak optimal karena pencariannya hanya bergantung pada perkiraan heuristik $h(n)$ dan mengabaikan akumulasi biaya aktual $g(n)$. Sebaliknya, UCS, A*, dan IDA* secara konsisten berhasil menemukan solusi yang optimal, namun dengan jumlah iterasi yang berbeda-beda. Algoritma A* adalah yang paling efisien dengan jumlah iterasi terendah berkat fungsi evaluasi $f(n) = g(n) + h(n)$ yang membuat arah pencariannya jadi lebih "terarah". Sementara itu, UCS melakukan penelusuran yang lebih luas karena harus mengekskansi semua kemungkinan murni berdasarkan $g(n)$, dan IDA* mencatat jumlah iterasi tertinggi akibat mekanismenya yang harus mengulang pencarian dari awal setiap kali melonggarkan ambang batas (threshold).

7.4 Testcase 4 - Papan Sedang

7.4.1 Input 4

```

10 10
XXXXXXXXXX
XZ*****OX
X*XXXXX**X
X*X**2X**X
X*****X
X*****X
X***X***X
X***X***X
X0*XX***1X
XXXXXXXXXX
999 999 999 999 999 999 999 999 999 999

```

```

999 1 1 1 1 1 1 1 1 999
999 50 999 999 999 999 999 1 1 999
999 50 999 1 1 1 999 1 1 999
999 50 1 1 1 1 1 1 1 999
999 50 1 1 1 1 1 1 1 999
999 50 1 1 999 1 1 1 1 999
999 1 1 1 999 1 1 1 1 999
999 1 1 999 999 1 1 1 1 999
999 999 999 999 999 999 999 999 999

```

7.4.2 Output UCS

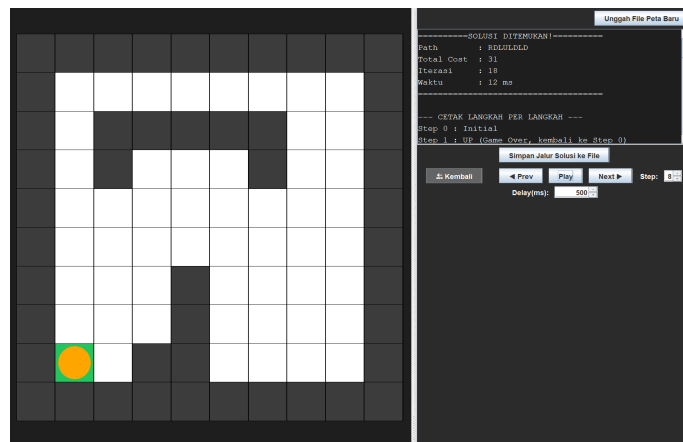


Figure 16: Hasil eksekusi UCS testcase 4

7.4.3 Output GBFS-H1

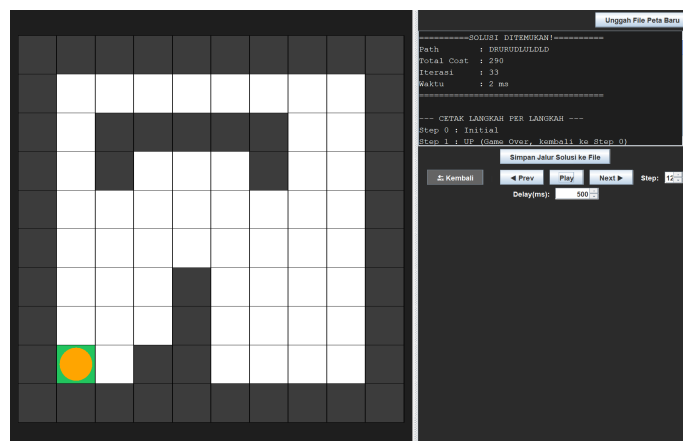


Figure 17: Hasil eksekusi GBFS-H1 testcase 4

7.4.4 Output GBFS-H2

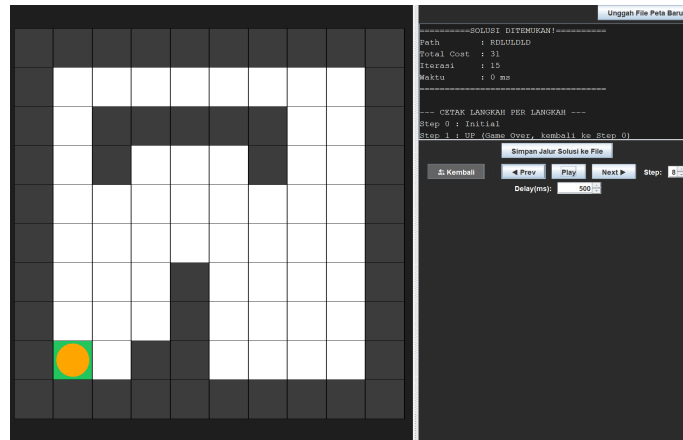


Figure 18: Hasil eksekusi GBFS-H2 testcase 4

7.4.5 Output GBFS-H3

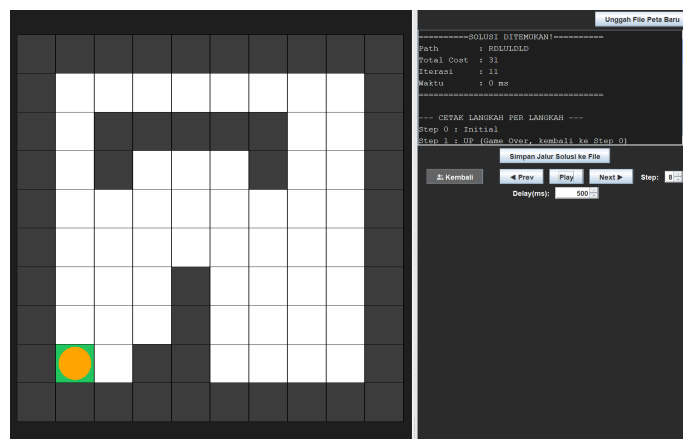


Figure 19: Hasil eksekusi GBFS-H3 testcase 4

7.4.6 Output A*-H1

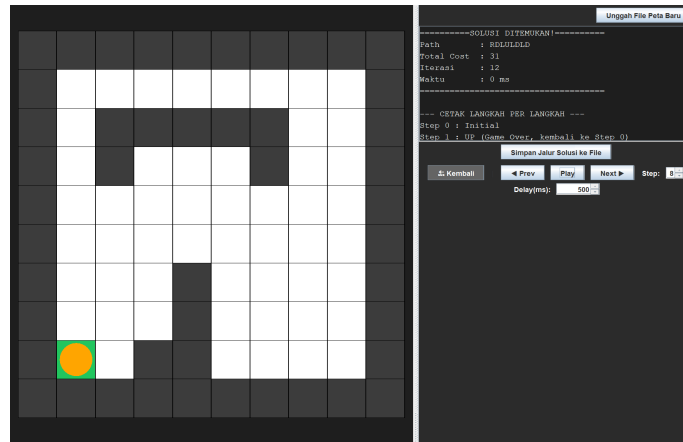


Figure 20: Hasil eksekusi A*-H1 testcase 4

7.4.7 Output A*-H2

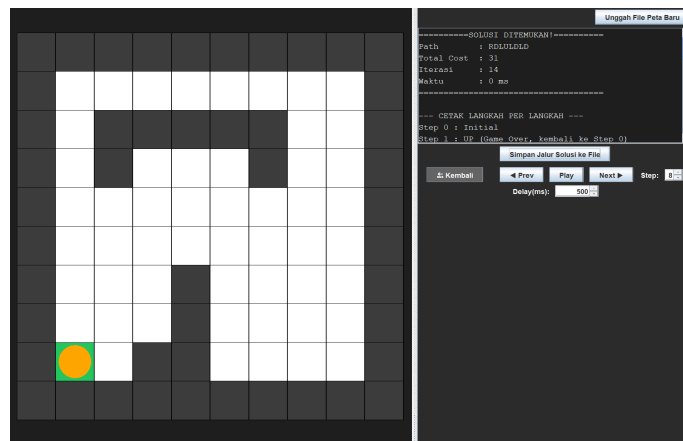


Figure 21: Hasil eksekusi A*-H2 testcase 4

7.4.8 Output A*-H3

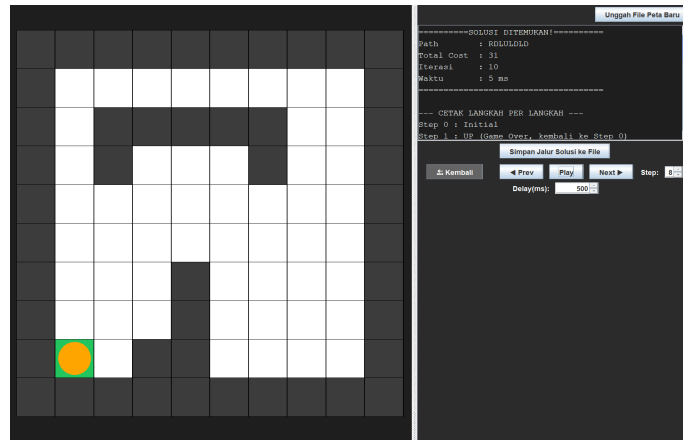


Figure 22: Hasil eksekusi A*-H3 testcase 4

7.4.9 Output IDA*-H1

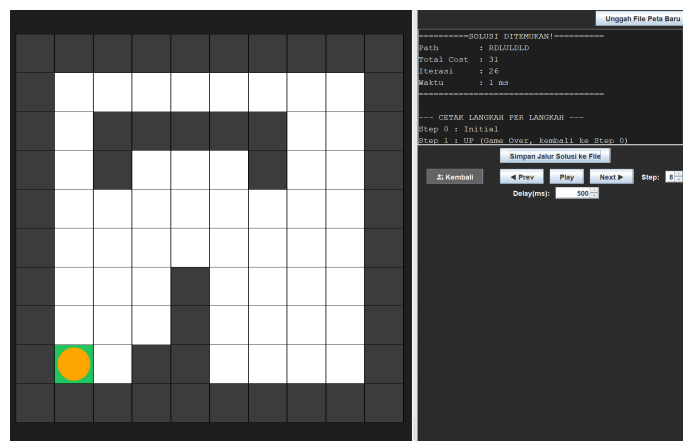


Figure 23: Hasil eksekusi IDA*-H1 testcase 4

7.4.10 Output IDA*-H2

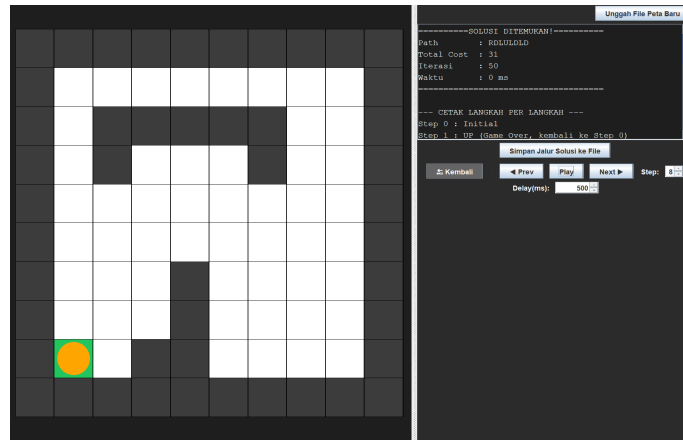


Figure 24: Hasil eksekusi IDA*-H2 testcase 4

7.4.11 Output IDA*-H3

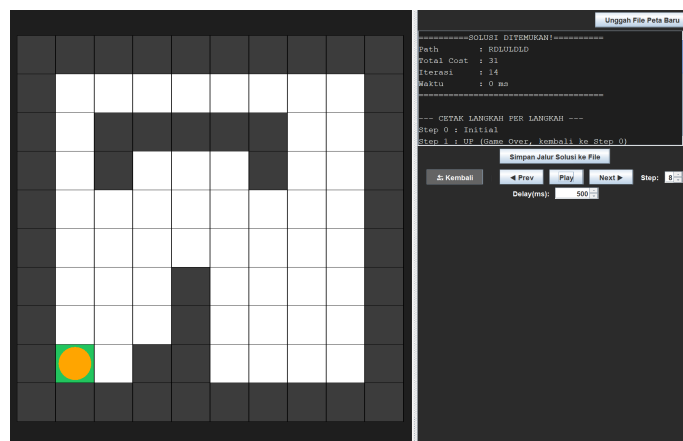


Figure 25: Hasil eksekusi IDA*-H3 testcase 4

7.4.12 Perbandingan Hasil

Table 4: Hasil Pengujian TC4

Algoritma	Heuristik	Cost	Iterasi	Waktu (ms)
UCS	-	31	18	12
GBFS	H1	290	33	2
GBFS	H2	31	15	0..
GBFS	H3	31	11	0..
A*	H1	31	12	0..
A*	H2	31	14	0..
A*	H3	31	15	5
IDA*	H1	31	26	1
IDA*	H2	31	50	0..
IDA*	H3	31	14	0..

7.4.13 Analisis

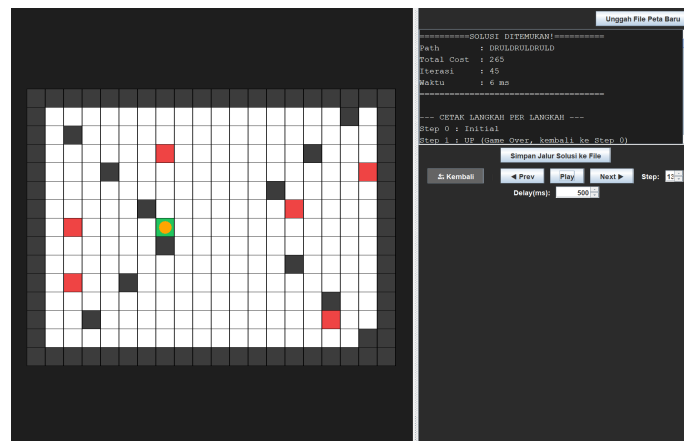
Berdasarkan hasil pengujian, terdapat dua fenomena menarik. Pertama, algoritma GBFS dengan heuristik Manhattan to Goal (H1) menghasilkan cost yang sangat tidak optimal akibat sifat butanya yang hanya mengandalkan kedekatan nilai tebakan $h(n)$. Algoritma "terpancing" untuk langsung mengejar target goal akhir, lalu terjebak berputar-putar di area tersebut demi mencari tile angka wajib yang terlewat, sehingga cost aktual penelusurannya membengkak. Kegagalan ini tidak terjadi pada H2 dan H3 yang bekerja secara sekuensial, sehingga algoritma GBFS pun tetap mampu bekerja dengan optimal tanpa terjebak.

Kedua, terjadi lonjakan iterasi yang signifikan pada algoritma IDA* dengan heuristik H2, yang mengindikasikan bahwa penelusuran cabang pada setiap threshold cukup dalam (mendekati titik akhir) sebelum akhirnya terpotong. Tingginya angka tersebut merupakan akumulasi dari setiap pelonggaran threshold. Sementara itu, jumlah step yang tercatat jauh lebih sedikit karena di setiap awal threshold, histori step akan direset sehingga log hanya menampilkan step-step dari threshold terakhir yang berhasil tuntas saja.

7.5 Testcase 5 - Papan Besar

7.5.1 Input 5

```
15 20
XXXXXXXXXXXXXXXXXXXXX
XZ*****X*X
X*X*****2*****X
X*****L*****X***X
X***X*****LX
X*****X*5***X
```

[illegible]

7.5.3 Output GBFS-H1

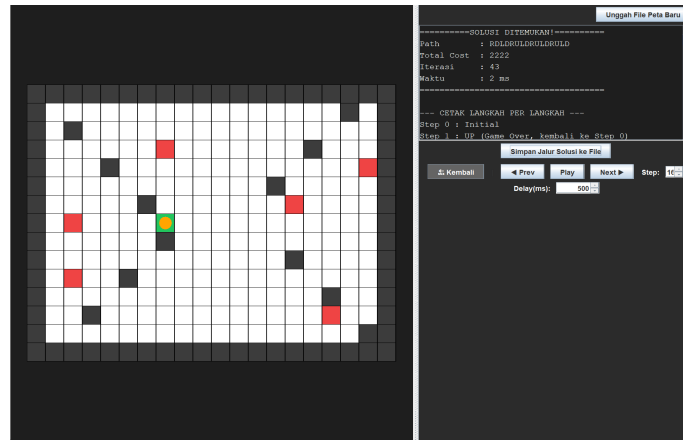


Figure 27: Hasil eksekusi GBFS-H1 testcase 5

7.5.4 Output GBFS-H2

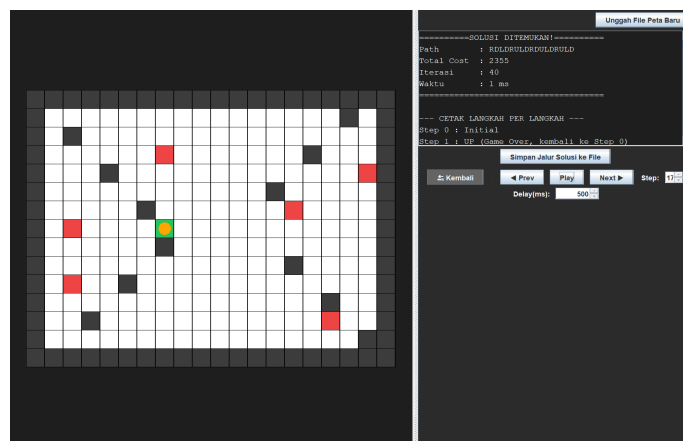


Figure 28: Hasil eksekusi GBFS-H2 testcase 5

7.5.5 Output GBFS-H3

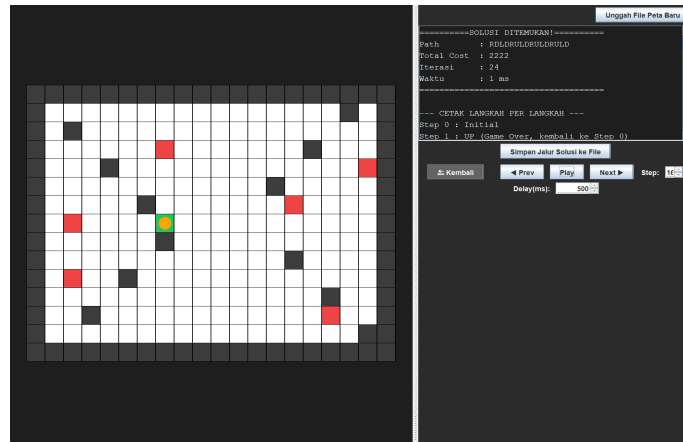


Figure 29: Hasil eksekusi GBFS-H3 testcase 5

7.5.6 Output A*-H1

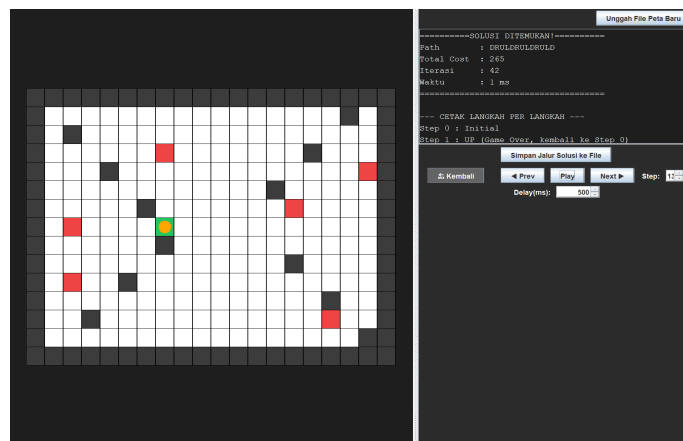


Figure 30: Hasil eksekusi A*-H1 testcase 5

7.5.7 Output A*-H2

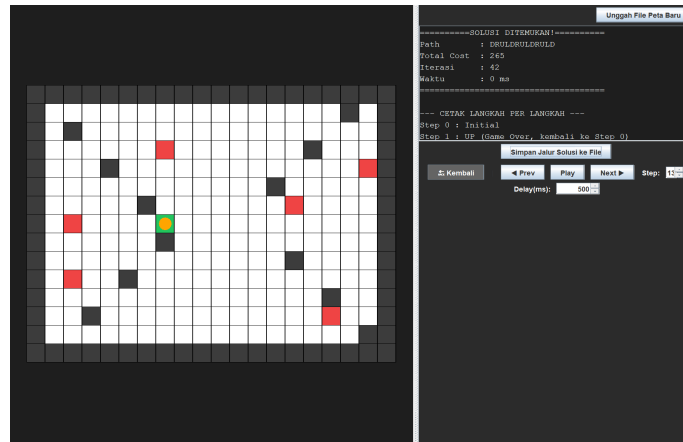


Figure 31: Hasil eksekusi A*-H2 testcase 5

7.5.8 Output A*-H3

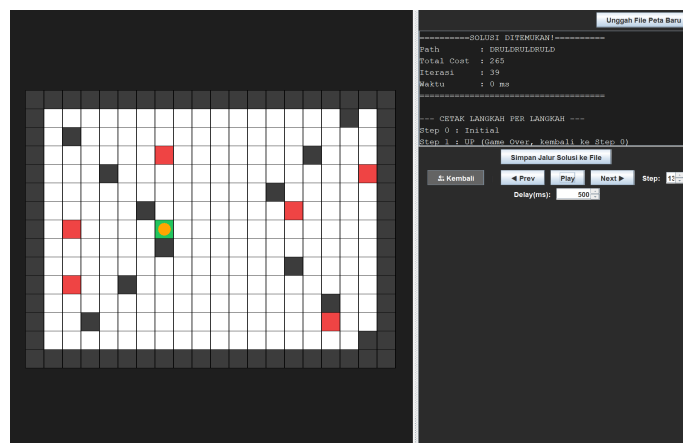


Figure 32: Hasil eksekusi A*-H3 testcase 5

7.5.9 Output IDA*-H1

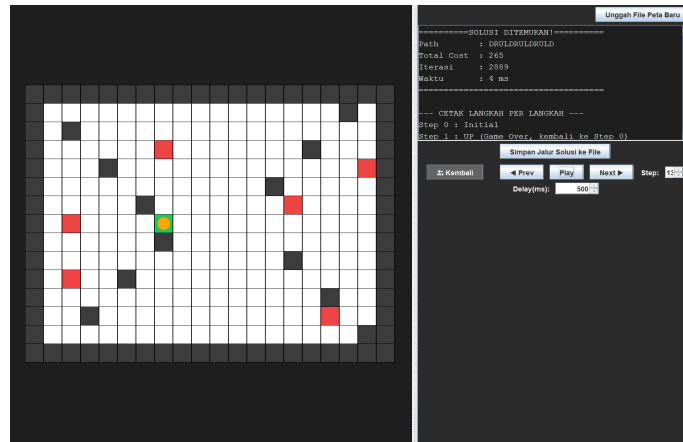


Figure 33: Hasil eksekusi IDA*-H1 testcase 5

7.5.10 Output IDA*-H2

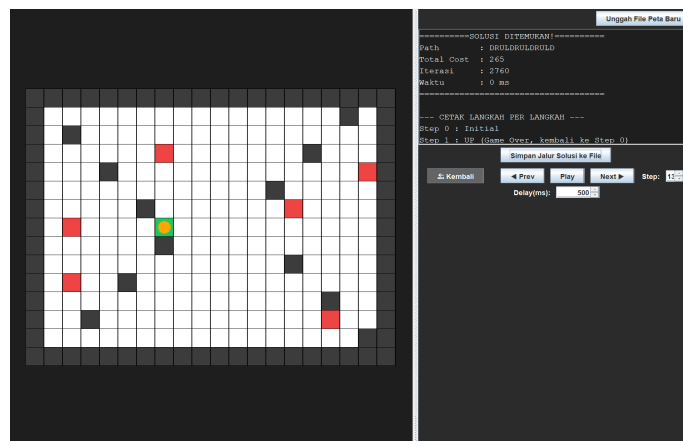


Figure 34: Hasil eksekusi IDA*-H2 testcase 5

7.5.11 Output IDA*-H3

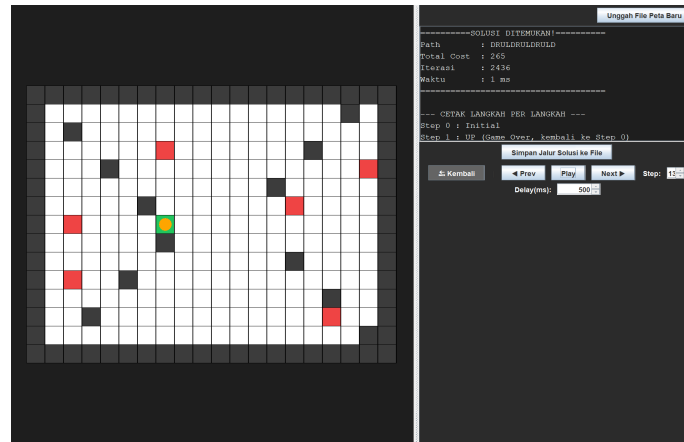


Figure 35: Hasil eksekusi IDA*-H3 testcase 5

7.5.12 Perbandingan Hasil

Table 5: Hasil Pengujian TC5

Algoritma	Heuristik	Cost	Iterasi	Waktu (ms)
UCS	-	265	45	6
GBFS	H1	2222	43	2
GBFS	H2	2355	40	1
GBFS	H3	2222	24	1
A*	H1	265	42	1
A*	H2	265	42	0..
A*	H3	265	39	0..
IDA*	H1	265	2889	4
IDA*	H2	265	2760	0..
IDA*	H3	265	2436	1

7.5.13 Analisis

Berdasarkan hasil pengujian, algoritma GBFS terbukti gagal mencapai solusi optimal akibat sifatnya yang mengabaikan biaya aktual $g(n)$, sebaliknya UCS, A*, dan IDA* yang secara konsisten berhasil menemukan cost terendah yaitu 265. Dari sisi efisiensi penelusuran, heuristik H3 tampil sebagai fungsi evaluasi yang paling akurat karena sukses iterasinya selalu minimum di setiap kelas algoritma. Algoritma IDA* lagi-lagi harus memiliki iterasi tertinggi akibat mekanisme pengulangan penelusuran dari awal setiap kali thresholdnya dinaikkan.

7.6 Testcase 6 - Impossible Case

7.6.1 Input 6

```
7 7
XXXXXXX
XZL***X
X*L***X
X*****X
X*****X
X*****X
X*****X
X*XXXXX
999 999 999 999 999 999 999
999  1 999  1  1  1 999
999 999 999  1  1  1 999
999  1  1  1  1  1 999
999  1  1  1  1  1 999
999  1  1  1  1  1 999
999 999 999 999 999 999 999
```

7.6.2 Output UCS

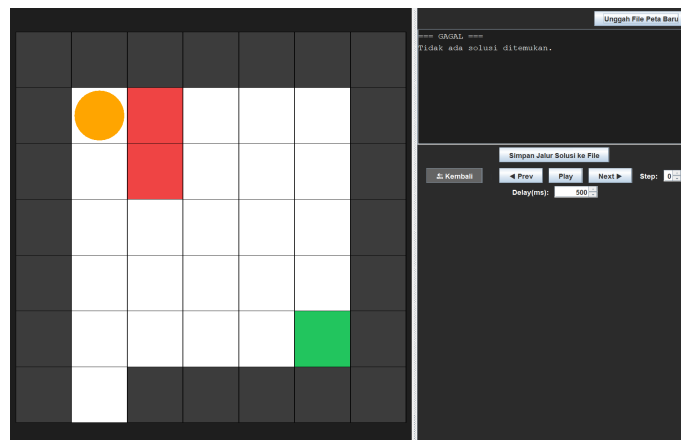


Figure 36: Hasil eksekusi UCS testcase 6

7.6.3 Output GBFS-H1

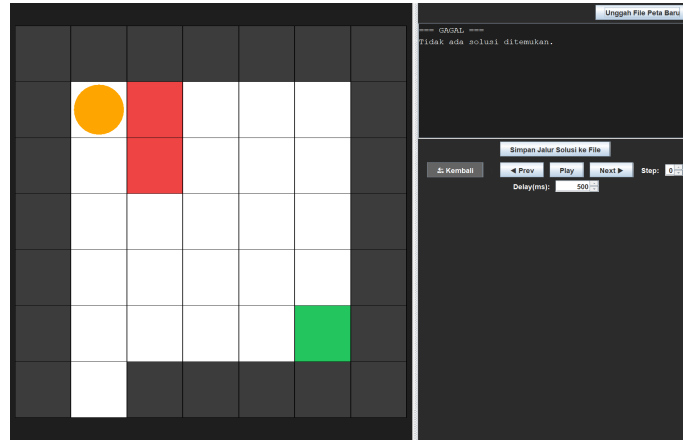


Figure 37: Hasil eksekusi GBFS-H1 testcase 6

7.6.4 Output GBFS-H2

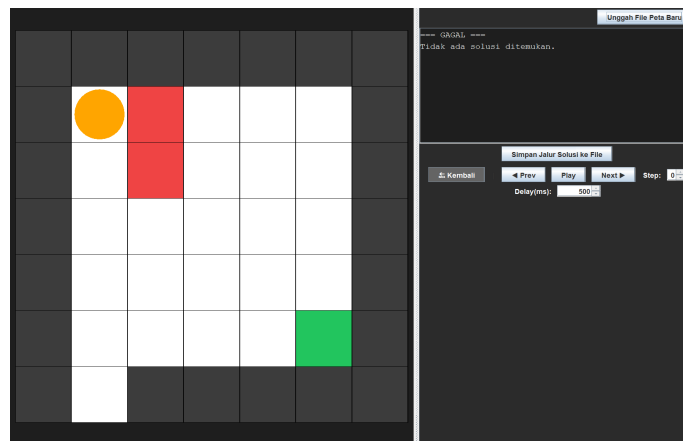


Figure 38: Hasil eksekusi GBFS-H2 testcase 6

7.6.5 Output GBFS-H3

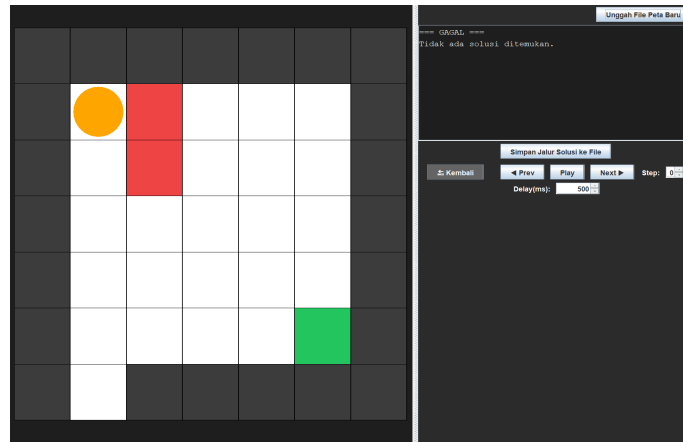


Figure 39: Hasil eksekusi GBFS-H3 testcase 6

7.6.6 Output A*-H1

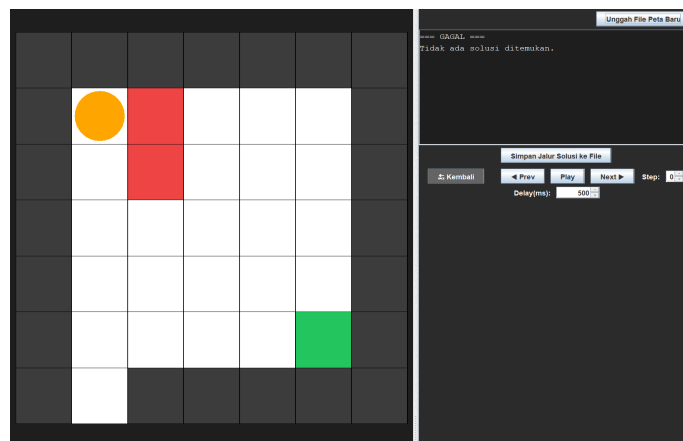


Figure 40: Hasil eksekusi A*-H1 testcase 6

7.6.7 Output A*-H2

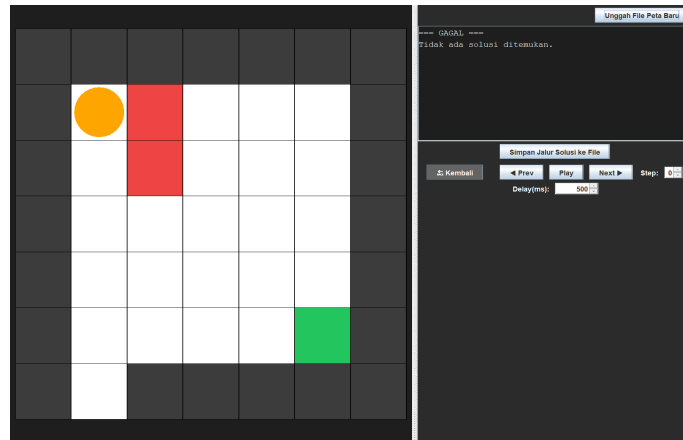


Figure 41: Hasil eksekusi A*-H2 testcase 6

7.6.8 Output A*-H3

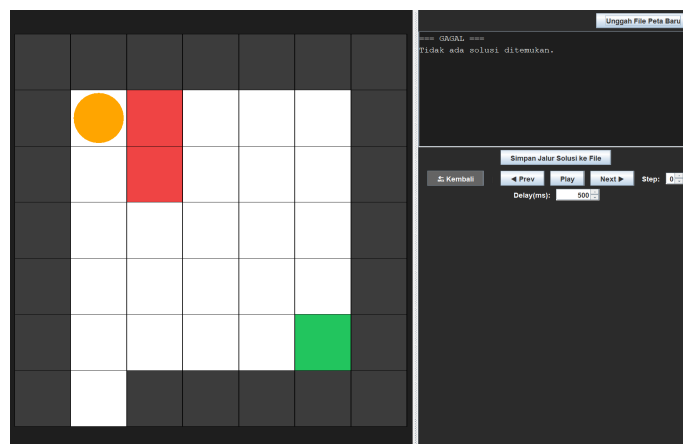


Figure 42: Hasil eksekusi A*-H3 testcase 6

7.6.9 Output IDA*-H1

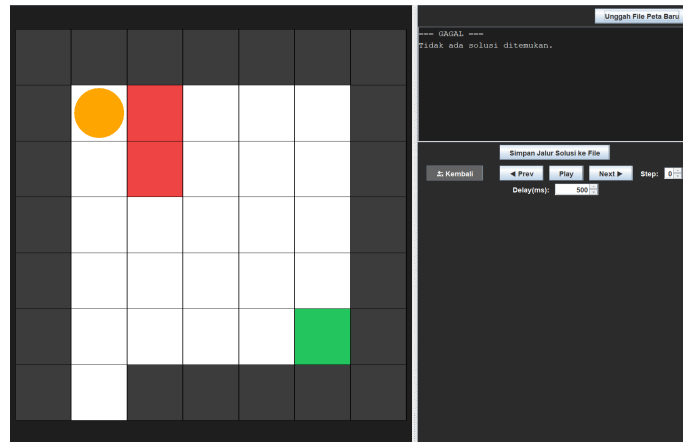


Figure 43: Hasil eksekusi IDA*-H1 testcase 6

7.6.10 Output IDA*-H2

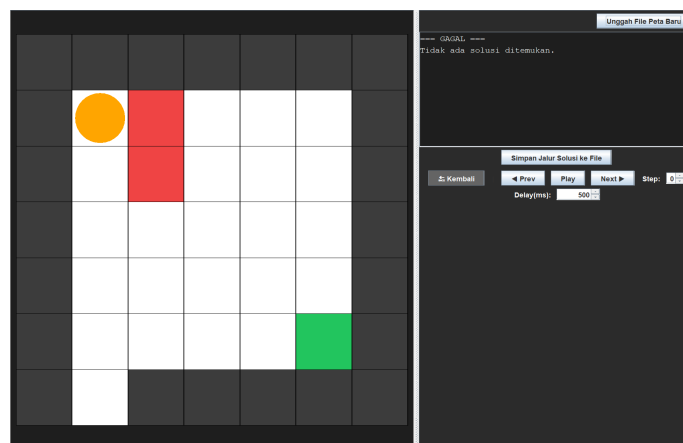


Figure 44: Hasil eksekusi IDA*-H2 testcase 6

7.6.11 Output IDA*-H3

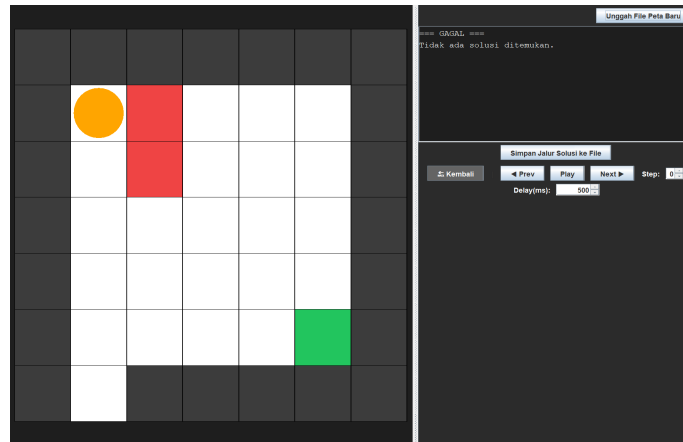


Figure 45: Hasil eksekusi IDA*-H3 testcase 6

7.6.12 Analisis

Berdasarkan hasil pengujian, seluruh algoritma dan heuristik tidak dapat mendapatkan solusi. Hal ini masuk akal karena papan memang dirancang demikian. Dari posisi semula hanya ada dua arah gerak yang valid, yaitu kiri dan bawah. Namun, apabila ke kiri akan menginjak lava dan apabila ke bawah akan keluar papan sehingga tidak ada solusi yang valid.

7.7 Analisis Akhir

Berdasarkan hasil pengujian, terdapat beberapa perbedaan karakteristik antar algoritma dan heuristik.

- UCS cenderung menghasilkan solusi optimal, tetapi jumlah iterasi dan penggunaan memorinya besar karena tidak menggunakan heuristic function.
- GBFS memiliki jumlah iterasi minimum, namun solusi yang dihasilkan tidak selalu optimal karena hanya mempertimbangkan nilai heuristik.
- A* memberikan keseimbangan terbaik antara kualitas solusi dan efisiensi pencarian karena mempertimbangkan traversal cost dan heuristic estimate secara bersamaan.
- IDA* memiliki penggunaan memori lebih kecil dibanding A*, tetapi pada beberapa testcase membutuhkan iterasi lebih banyak akibat proses pencarian ulang pada setiap thresholdnya.

Pengaruh heuristic function juga terlihat pada jumlah iterasi yang dilakukan algoritma.

- Heuristik H1 (Manhattan to Goal) memiliki perhitungan paling sederhana, tetapi kurang memperhatikan urutan checkpoint sehingga dapat "tergocek".
- Heuristik H2 (Manhattan to Next Target) menghasilkan pencarian lebih terarah karena mempertimbangkan objective aktif berikutnya.
- Heuristik H3 (Manhattan Total to Goal) memberikan estimasi paling lengkap karena memperhitungkan seluruh checkpoint hingga goal, namun biaya perhitungan heuristik menjadi lebih besar.

Pada testcase dengan banyak checkpoint, heuristik H2 dan H3 umumnya menghasilkan jumlah iterasi lebih sedikit dibanding H1 karena pencarian menjadi lebih fokus terhadap urutan objective permainan.

A Lampiran

A.1 Link Repository

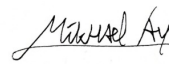
https://github.com/mhps-null/Tucil3_13524051_13524053.git

A.2 Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Muhammad Haris Putra Sulastianto



Mikhael Andrian Yonatan

A.3 Tabel Spesifikasi Tugas Kecil

Table 6: Checklist Spesifikasi Program

No	Poin	Ya	Tidak
1	Program berhasil dikompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	✓	
4	Program dapat membaca masukan berkas <code>.txt</code> serta menyimpan solusi dalam berkas <code>.txt</code>	✓	
5	Program memiliki Graphical User Interface (GUI)	✓	
6	Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (IDA*)	✓	
7	Ada tambahan dua alternatif heuristik selain yang utama (Manhattan to Next Target dan Manhattan Total to Goal)	✓	